Product Requirements Document (PRD)

GymBro AI

AI-Powered Fitness & Nutrition Platform

Version: 1.0
Owner: You
Tech Stack: React + Spring Boot + Supabase PostgreSQL + Redis + Kafka + Docker + AI

---

1. Vision

GymBro AI is an intelligent fitness ecosystem that goes beyond workout logging. It provides personalized workout plans, AI-driven nutrition coaching, food recognition, progress analytics, and habit tracking while maintaining production-grade backend architecture.

Unlike a chatbot, GymBro AI understands a user's history and provides contextual recommendations.

---

2. Objectives

User Objectives

- Track workouts effortlessly
- Track nutrition accurately
- Scan food instead of manual entry
- Receive intelligent recommendations
- Visualize long-term progress
- Stay consistent

Technical Objectives

- Production-grade backend

- Scalable architecture
- Secure authentication
- Distributed systems
- Event-driven services
- Resume-worthy project

---

3. Target Users

Beginner

Needs workout guidance.

Intermediate

Needs progressive overload tracking.

Advanced

Needs analytics and optimization.

---

4. Success Metrics

User Metrics

- Daily Active Users
- Workout completion rate
- Food scan accuracy
- Weekly retention

Technical Metrics

- API response <200ms
- Dashboard load <500ms
- Redis cache hit >80%
- 99% API uptime

---

5. Tech Stack

Frontend

- React
- TypeScript
- Redux Toolkit
- React Query
- Tailwind CSS
- React Router

Backend

- Java 21
- Spring Boot
- Spring Security
- JWT Authentication
- Spring Data JPA
- Flyway

Database

Supabase PostgreSQL

Cache

Redis

Event Streaming

Kafka

AI

- LLM
- Vision Model
- OCR

DevOps

- Docker
- Docker Compose
- GitHub Actions

---

6. MVP Features

Module 1 — Authentication

Features

- Register
- Login
- Logout
- Refresh Token
- Forgot Password
- Reset Password
- Email Verification
- Change Password

Security

- JWT
- BCrypt
- Refresh Tokens
- Rate Limiting
- Input Validation

Module 2 — User Profile

Fields

- Name
- Age
- Height
- Weight
- Gender
- Fitness Goal
- Activity Level
- Experience Level

Module 3 — Workout Planner

Features

Exercise Library

Workout Templates

Push Pull Legs

Upper Lower

Bro Split

Custom Workouts

Progressive Overload

Workout History

Rest Timer

PR Tracking

Exercise Notes

Module 4 — Nutrition

Manual Meal Logging

Daily Calories

Protein

Carbs

Fat

Water Intake

Favorite Foods

Recent Foods

Meal History

Module 5 — Food Scanner

Upload Image

↓

Vision AI

↓

Detect Food

↓

Estimate Quantity

↓

Calculate Nutrition

↓

Save Meal

Module 6 — AI Coach

Instead of chatting...

The AI should understand:

Current Goal

Workout History

Calories

Weight Trend

Sleep

Protein Intake

Recovery

Then generate:

Workout modifications

Diet recommendations

Recovery advice

Progress analysis

Module 7 — Dashboard

Daily Calories

Today's Workout

Protein Goal

Water Goal

Sleep

Weight

Weekly Consistency

Workout Streak

Module 8 — Progress

Weight Graph

Strength Graph

Volume Graph

Calories Graph

Body Measurements

Weekly Summary

Monthly Summary

Module 9 — Notifications

Workout Reminder

Water Reminder

Meal Reminder

Achievement

Goal Completed

Kafka will power these events.

---

7. Redis Usage

Dashboard Cache

Exercise Cache

User Session Cache

OTP Cache

Rate Limiting

Leaderboard Cache

Frequently Used Foods

---

8. Kafka Usage

Events

Workout Completed

↓

Update Analytics

↓

Generate Achievement

↓

Send Notification

↓

Update Dashboard

Each service consumes events independently.

---

9. Database Schema

users

- id

- email
- password
- created_at

---

profiles

- user_id
- age
- weight
- height
- goal

---

exercises

- id
- name
- muscle_group
- equipment
- instructions

---

workouts

- id
- user_id
- title

---

workout_sets

- workout_id
- exercise_id
- weight
- reps

---

foods

- id
- name
- calories
- protein
- carbs
- fat

---

meals

- id
- user_id
- food_id
- quantity

---

body_measurements

- user_id
- weight
- body_fat
- chest
- waist

---

notifications

- id
- user_id
- title
- type

---

refresh_tokens

- token

- expiry

---

10. APIs

Authentication

POST /auth/register

POST /auth/login

POST /auth/refresh

POST /auth/logout

POST /auth/forgot-password

POST /auth/reset-password

User

GET /profile

PUT /profile

Workout

GET /workouts

POST /workouts

PUT /workouts

DELETE /workouts

Nutrition

POST /meals

GET /meals

DELETE /meals

Food Scan

POST /food/scan

Dashboard

GET /dashboard

Analytics

GET /analytics

AI

POST /ai/recommend

POST /ai/workout

POST /ai/nutrition

---

11. Folder Structure

gymbro-ai/

frontend/

backend/

docker/

docs/

postman/

architecture/

scripts/

README.md

---

12. Stretch Features (V2)

- Social feed
- Friends
- Challenges
- Leaderboards
- Wearable integration (Apple Health/Google Fit)
- Smartwatch workout sync
- Voice AI coach
- Calendar integration
- Smart grocery list
- Recipe generation
- Supplement recommendations
- Injury-aware workout planning

---

13. Non-Functional Requirements

Performance

- Dashboard loads in <500 ms for cached data.
- API p95 response time <200 ms for common endpoints.
- Support at least 1,000 concurrent users in testing.

Security

- JWT with refresh tokens.
- Passwords hashed using BCrypt.
- Role-based authorization (User/Admin).
- Input validation and SQL injection protection.
- Secure HTTP headers and CORS configuration.

Reliability

- Centralized exception handling.
- Structured logging.
- Database migrations managed with Flyway.
- Automated backups for PostgreSQL.

Testing

- Unit tests for services.
- Integration tests for APIs.
- Frontend component tests.
- End-to-end smoke tests for core user flows.

---

14. Resume Impact

This project is designed to demonstrate experience with:

- Full-stack application development using React and Spring Boot
- Secure authentication and authorization
- PostgreSQL database design and optimization
- Redis caching strategies
- Event-driven architecture with Kafka
- AI-powered recommendation systems and food recognition
- RESTful API design
- Docker-based containerization
- CI/CD and production-oriented engineering practices
- Scalable software architecture and clean code principles

This is intentionally scoped to resemble a modern SaaS product rather than a portfolio CRUD application, making it a strong talking point for software engineering interviews.